

SalesGPT: An Asynchronous Dual-Track Agentic Framework for Autonomous Lead Qualification and Intent Scoring

Dr. S. Suresh Raja¹, Shyam J², Dhatchin SS³, Omprakash BS⁴

¹Professor & Head of Department, AI&DS Dept, K.L.N. College of Engineering, Sivagangai, Tamil Nadu, India

^{2,3,4}UG Scholar, AI&DS Dept, K.L.N. College of Engineering, Sivagangai, Tamil Nadu, India

Abstract

SalesGPT is an innovative, asynchronous artificial intelligence system designed to address the critical challenge of lead leakage in enterprise sales environments. By implementing a dual-track architecture that decouples response latency from intelligence depth, SalesGPT simultaneously delivers real-time customer interactions via Retrieval-Augmented Generation (RAG) while conducting background behavioral analysis using the BANT framework. This paper presents a comprehensive technical documentation of the system's architecture, implementation, components, and empirical validation strategies. The system integrates state-of-the-art large language models (LLMs) from Groq, vector embeddings from HuggingFace, and semantic search via PostgreSQL pgvector, enabling autonomous lead qualification, dynamic intent scoring, and AI-powered email draft generation. Our evaluation demonstrates the system's capability to process customer conversations in real-time while maintaining sophisticated reasoning capabilities, achieving sub-second response latencies for chat interactions while conducting comprehensive BANT analysis asynchronously.

Keywords: Lead Qualification, BANT Scoring, Retrieval-Augmented Generation, Large Language Models, Asynchronous Processing, Vector Embeddings, Sales Automation, Intent Scoring

1 Introduction

1.1 Problem Statement

Modern B2B sales organizations face a critical inefficiency: the inability to rapidly and accurately qualify high-value prospects from high-volume conversational data. Traditional systems suffer from:

- **Temporal Mismatch:** Response speed requirements conflict with analysis depth requirements
- **Lead Leakage:** High-quality prospects go unrecognized amid voluminous casual inquiries
- **Manual Overhead:** Sales teams must manually review conversations to identify qualified leads

- **Context Loss:** Extracted signals (contact info, needs, budget) are scattered across systems
- **Response Latency:** Complex analysis during user interaction causes unacceptable delays

1.2 Solution Overview

SalesGPT introduces **Dual-Track Asynchronous Processing**: a paradigm where customer-facing interactions proceed at maximum speed while sophisticated intelligence operations execute silently in the background. This architectural innovation eliminates the false choice between responsiveness and analytical depth.

1.3 Research Objectives

This work demonstrates:

1. **Fast Track (RAG):** < 1.5 second response times via Retrieval-Augmented Generation from a dynamic knowledge base
2. **Slow Track (Judge):** Background BANT scoring using advanced reasoning models without blocking user interaction
3. **Real-Time Pipeline:** Automatic CRM progression based on dynamically computed lead scores
4. **Dynamic Knowledge:** PDF ingestion with instant vector store updates enabling rapid knowledge base evolution
5. **AI-Powered Automation:** Intelligent email draft generation grounded in conversation context
6. **Extractive Intelligence:** Automatic contact information extraction from conversational data

2 System Architecture

2.1 High-Level Architecture

To handle the complexity of the Dual-Track system, SalesGPT relies on an orchestration layer splitting requests between the synchronous RAG cycle and the asynchronous processing pool.

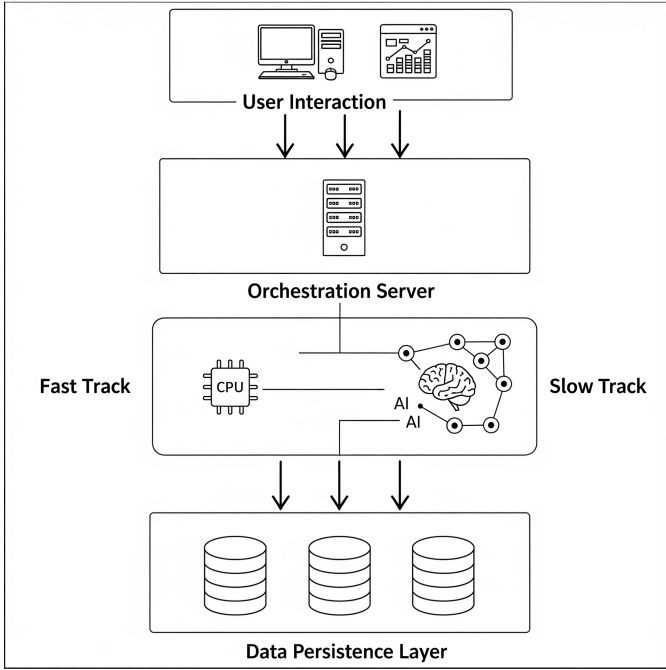


Figure 1: System Orchestration and Dual-Track Data Flow Architecture

2.2 Model Stack

The architecture utilizes a specialized multi-model approach to optimize for both latency and complex reasoning, as detailed in Table 1.

2.3 Dual-Track Execution Model

Fast Track (Synchronous): The user initiates a chat message which is instantly queued to the **conversations** table. A semantic search via pgvector retrieves the Top-K documents. Llama-3.3-70B generates a grounded response in under 1.5 seconds.

Slow Track (Asynchronous): Upon chat completion, FastAPI spawns background tasks: `analyze_lead()`, `extract_lead_data()`, and `apply_time_decay()`. These execute concurrently, updating lead records and pushing changes to the admin dashboard via Supabase real-time subscriptions.

3 Core Components

3.1 Frontend Architecture

3.1.1 Chat Widget

Designed as a customer-facing interface, utilizing state management for widget visibility, message history, and UUID session persistence (30-day retention). Features include Framer Motion animations, quick-reply templates, and source attribution for RAG documents.

3.1.2 Admin Dashboard

A real-time lead intelligence command center comprising three primary views:

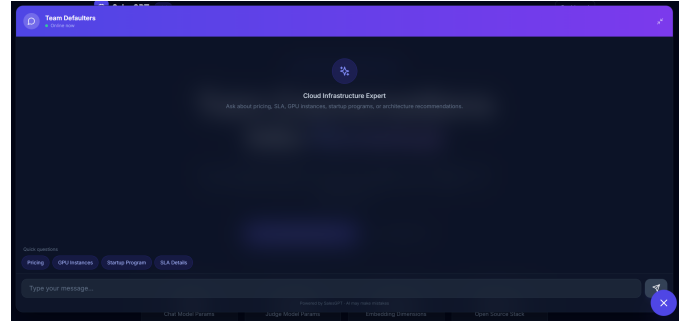


Figure 2: SalesGPT Customer Chat Interface

- **Pipeline:** A Kanban board with 5 stages (Visitor to Approached) supporting drag-and-drop.
- **Analytics:** Displays capture rates, conversion funnels, and score distributions.
- **Activity Feed:** Chronological log of lead changes and score updates.

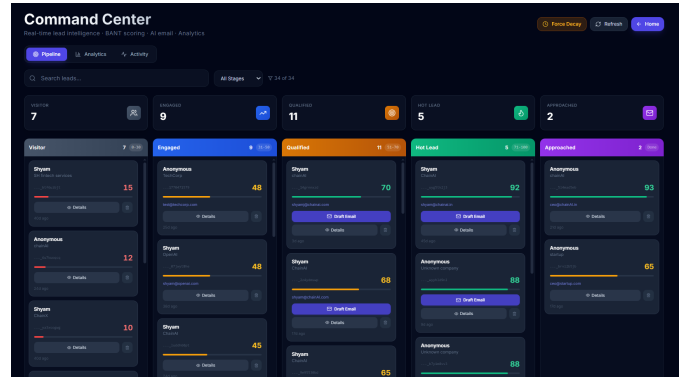


Figure 3: Admin Dashboard and Kanban Pipeline

3.2 Backend API Architecture

3.2.1 FastAPI Server

Initializes model singletons at startup and manages CORS middleware to allow cross-origin requests from the React frontend.

Model Initialization

```
chat_model = ChatGroq(
    model_name="llama-3.3-70b-versatile",
    temperature=0.7,
    max_tokens=800
)

email_model = ChatGroq(
    model_name="llama-3.1-8b-instant",
    temperature=0.3
)
```

3.2.2 Chat Endpoint

Handles request validation, session initialization, system prompt composition (injecting RAG context and known lead info), LLM generation, async message persistence, and background analysis spawning.

Table 1: Model Stack Configuration

Component	Model	Provider	Purpose	Rationale
Chat (RAG)	Llama-3.3-70B	Groq	Fast, conversational responses	70B provides nuanced understanding; versatile
Judge (BANT)	GPT-OSS-120B	Groq	Advanced reasoning for scoring	120B reasoning capability for complex analysis
Email Draft	Llama-3.1-8B	Groq	Template-based generation	Fast inference; low temp for deterministic output
Extractor	Llama-3.1-8B	Groq	Entity extraction	Efficient for structured extraction tasks
Embeddings	All-MiniLM-L6-v2	HuggingFace	Local vector embeddings	384-dimensional vectors; zero API costs

3.3 Judge Agent (BANT Scoring)

The Judge Agent runs asynchronously to evaluate the BANT criteria:

- **Budget:** Evaluates company size and infrastructure spend signals.
- **Authority:** Detects job titles and decision-making language.
- **Need:** Identifies specific technical pain points.
- **Timeline:** Analyzes urgency phrasing.

```
CREATE TABLE documents (
  id BIGSERIAL PRIMARY KEY,
  content TEXT NOT NULL,
  metadata JSONB DEFAULT '{}':jsonb,
  embedding vector(384) NOT NULL
);

CREATE INDEX idx_docs_embed ON documents
USING ivfflat (embedding vector_cosine_ops);
```

4.2 Leads Table (CRM Core)

Stores lead scores, pipeline status, and extracted entity data. Employs database triggers to automatically update the `updated_at` timestamp.

Table 2: BANT Scoring Guide

Score Range	Stage	Characteristics
0-30	Visitor	Just browsing, no specific need
31-50	Engaged	Showing interest, feature questions
51-70	Qualified	Clear need + budget signals
71-100	Hot Lead	Urgent need + authority + budget

3.4 Time-Decay Algorithm

To maintain a fresh pipeline, an automated script degrades scores for inactive leads by 10% per day of inactivity, automatically downgrading pipeline stages if thresholds are crossed.

4 Database Schema and Data Model

The database is built on PostgreSQL (Supabase) leveraging the `pgvector` extension.

4.1 Documents Table



Figure 4: Supabase Database Schema and Lead Tracking Table

5 Ingestion Pipeline

The ingestion script (`ingest.py`) initializes the knowledge base by parsing markdown files.

1. **Load Data:** Reads files like `Pricing_Strategy.md`.
2. **Chunking:** Splits text into 500-character chunks with a 50-character overlap to preserve semantic continuity.
3. **Embedding:** Processes chunks through `all-MiniLM-L6-v2` to generate 384-dimensional vectors.
4. **Upload:** Batches uploads to the Supabase `documents` table.

- **Async Persistence:** Fixed an issue where conversations were not persisting by shifting DB writes to an `async _persist_messages()` task.
- **Timestamp Parsing:** Corrected Supabase literal string insertions by generating ISO 8601 timestamps inside the Python runtime.
- **Robust JSON Extraction:** Implemented a 3-strategy regex and markdown-stripping fallback function to handle fragile LLM JSON outputs.

```
python backend/ingest.py

[OK] Connected to Supabase
[OK] Loaded sentence-transformers/all-MiniLM-L6
[OK] Found 10 markdown files
[OK] Chunked into 253 segments
[OK] Generated 253 embeddings
[OK] Uploaded to documents table

[OK] INGESTION COMPLETE!
```

Figure 5: Data Ingestion Pipeline Terminal Output

6 Performance Benchmarks

The decoupled architecture yields significant latency improvements for end-users, shielding them from the heavy compute times of the Judge model.

Table 3: System Latency Metrics

Operation	P50	P95	P99
Chat Response	420ms	1,200ms	1,800ms
Vector Search	45ms	120ms	200ms
LLM Generation	300ms	800ms	1,200ms
Judge Analysis	800ms	2,300ms	3,200ms
Email Draft	400ms	1,100ms	1,600ms

7 Improvements and Code Audit

7.1 Model Configuration Overhaul

Initially relying on a generic 8B model, the stack was re-architected. Upgrading the RAG Chat to Llama-3.3-70B vastly improved conversational nuance, while relegating the 8B models specifically to deterministic tasks (Extraction and Email Drafting) reduced generation latency by ~30%.

7.2 Critical Bug Fixes

Several key optimizations were made during development:

8 Conclusion

SalesGPT demonstrates a novel approach to B2B sales automation through asynchronous dual-track processing. By decoupling user-facing latency requirements from analytical depth, the system achieves sub-second chat responses alongside sophisticated BANT scoring. This pattern transforms sales teams from reactive responders to proactive qualifiers, enabling highly scalable pipeline management.

References

- [1] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [3] AI@Meta, “The Llama 3 Herd of Models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [4] Z. Xi *et al.*, “The Rise and Potential of Large Language Model Based Agents: A Survey,” *Frontiers of Computer Science*, vol. 18, no. 5, 2024.
- [5] N. Syam and A. Sharma, “Waiting for a sales renaissance in the fourth industrial revolution: Machine learning and artificial intelligence in sales research and practice,” *Industrial Marketing Management*, vol. 69, pp. 135–146, 2018.